

需强调的是，由于向量内部含有动态分配的空间，默认的运算符“=”不足以支持向量之间的直接赋值。例如，6.3节将以二维向量形式实现图邻接表，其主向量中的每一元素本身都是一维向量，故通过默认赋值运算符，并不能复制向量内部的数据区。

为适应此类赋值操作的需求，可如代码2.3所示，重载向量的赋值运算符。

```
1 template <typename T> Vector<T>& Vector<T>::operator=(Vector<T> const& V ) { //重载赋值操作符
2     if (_elem) delete [] _elem; //释放原有内容
3     copyFrom(V._elem, 0, V.size()); //整体复制
4     return *this; //返回当前对象的引用，以便链式赋值
5 }
```

代码2.3 重载向量赋值操作符

2.3.3 析构方法

与所有对象一样，不再需要的向量，应借助析构函数（**destructor**）及时清理（**cleanup**），以释放其占用的系统资源。与构造函数不同，同一对象只能有一个析构函数，不得重载。

向量对象的析构过程，如代码2.1中的方法~Vector()所示：只需释放用于存放元素的内部数组_elem[]，将其占用的空间交还操作系统。_capacity和_size之类的内部变量无需做任何处理，它们将作为向量对象自身的一部分被系统回收，此后既无需也无法被引用。

若不计系统用于空间回收的时间，整个析构过程只需 $O(1)$ 时间。

同样地，向量中的元素可能不是程序语言直接支持的基本类型。比如，可能是指向动态分配对象的指针或引用，故在向量析构之前应该提前释放对应的空间。出于简化的考虑，这里约定并遵照“谁申请谁释放”的原则。究竟应释放掉向量各元素所指的對象，还是需要保留这些对象，以便通过其它指针继续引用它们，应由上层调用者负责确定。

§ 2.4 动态空间管理

2.4.1 静态空间管理

内部数组所占物理空间的容量，若在向量的生命期内不允许调整，则称作静态空间管理策略。很遗憾，该策略的空间效率难以保证。一方面，既然容量固定，总有可能在此后的某一时刻，无法加入更多的新元素——即导致所谓的上溢（**overflow**）。例如，若使用向量来记录网络访问日志，则由于插入操作远多于删除操作，必然频繁溢出。注意，造成此类溢出的原因，并非系统不能提供更多的空间。另一方面反过来，即便愿意为降低这种风险而预留出部分空间，也很难在程序执行之前，明确界定一个合理的预留量。以上述copyFrom()方法为例，即便将容量取作初始规模的两倍，也只能保证在此后足够长的一段时间内（而并非永远）不致溢出。

向量实际规模与其内部数组容量的比值（即_size/_capacity），亦称作装填因子（**load factor**），它是衡量空间利用率的重要指标。从这一角度，上述难题可归纳为：

如何才能保证向量的装填因子既不致于超过1，也不致于太接近于0？

为此，需要改用动态空间管理策略。其中一种有效的方法，即使用所谓的可扩充向量。

2.4.2 可扩充向量

经过一段时间的生长，每当身体无法继续为其外壳所容纳，蝉就会蜕去外壳，同时换上一身更大的外壳。扩充向量（**extendable vector**）的原理，与之相仿。若内部数组仍有空余，则操作可照常执行。每经一次插入（删除），可用空间都会减少（增加）一个单元（图2.1(a)）。一旦可用空间耗尽（图(b)），就动态地扩大内部数组的容量。这里的难点及关键在于：

如何实现扩容？新的容量取作多少才算适宜？

首先解决前一问题。直接在原有物理空间的基础上追加空间？这并不现实。数组特有的定址方式要求，物理空间必须地址连续，而我们却无法保证，其尾部总是预留了足够空间可供拓展。

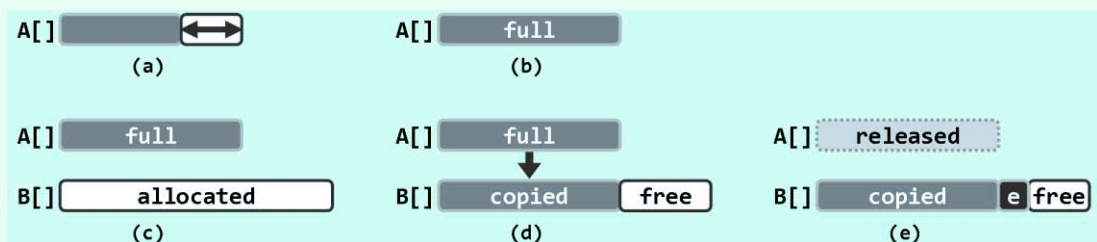


图2.1 可扩充向量的溢出处理

一种可行的方法，如图2.1(c~e)所示。我们需要另行申请一个容量更大的数组B[]（图(c)），并将原数组中的成员集体搬迁至新的空间（图(d)），此后方可顺利地插入新元素e而不致溢出（图(e)）。当然，原数组所占的空间，需要及时释放并归还操作系统。

2.4.3 扩容

基于以上策略的扩容算法expand()，可实现如代码2.4所示。

```
1 template <typename T> void Vector<T>::expand() { //向量空间不足时扩容
2     if (_size < _capacity) return; //尚未满员时，不必扩容
3     if (_capacity < DEFAULT_CAPACITY) _capacity = DEFAULT_CAPACITY; //不低于最小容量
4     T* oldElem = _elem; _elem = new T[_capacity <<= 1]; //容量加倍
5     for (int i = 0; i < _size; i++)
6         _elem[i] = oldElem[i]; //复制原向量内容 (T为基本类型，或已重载赋值操作符'=')
7     delete [] oldElem; //释放原空间
8 }
```

代码2.4 向量内部数组动态扩容算法expand()

实际上，在调用insert()接口插入新元素之前，都要先调用该算法，检查内部数组的可用容量。一旦当前数据区已满（_size == _capacity），则将原数组替换为一个更大的数组。

请注意，新数组的地址由操作系统分配，与原数据区没有直接的关系。这种情况下，若直接引用数组，往往会导致共同指向原数组的其它指针失效，成为野指针（wild pointer）；而经封装为向量之后，即可继续准确地引用各元素，从而有效地避免野指针的风险。

这里的关键在于，新数组的容量总是取作原数组的两倍——这正是上述后一问题的答案。

2.4.4 分摊分析

■ 时间代价

与常规数组实现相比，可扩充向量更加灵活：只要系统尚有可用空间，其规模将不再受限于初始容量。不过，这并非没有代价——每次扩容，元素的搬迁都需要花费额外的时间。

准确地，每一次由 n 到 $2n$ 的扩容，都需要花费 $O(2n) = O(n)$ 时间——这也是最坏情况下，单次插入操作所需的时间。表面看来，这一扩容策略似乎效率很低，但这不过是一种错觉。

请注意，按照此处的约定，每花费 $O(n)$ 时间实施一次扩容，数组的容量都会加倍。这就意味着，至少要再经过 n 次插入操作，才会因为可能溢出而再次扩容。也就是说，随着向量规模的不断扩大，在执行插入操作之前需要进行扩容的概率，也将迅速降低。故就某种平均意义而言，用于扩容的时间成本不至很高。以下不妨就此做一严格的分析。

■ 分摊复杂度

这里，不妨考查对可扩充向量的足够多次连续操作，并将其间所消耗的时间，分摊至所有的操作。如此分摊平均至单次操作的时间成本，称作分摊运行时间（**amortized running time**）。

请注意，这一指标与平均运行时间（**average running time**）有着本质的区别（习题[2-1]）。后者是按照某种假定的概率分布，对各种情况下所需执行时间的加权平均，故亦称作期望运行时间（**expected running time**）。而前者则要求，参与分摊的操作必须构成和来自一个真实可行的操作序列，而且该序列还必须足够地长。

相对而言，分摊复杂度可以针对计算成本和效率，做出更为客观而准确的估计。比如在这里，在任何一个可扩充向量的生命期内，在任何足够长的连续操作序列中，以任何固定间隔连续出现上述最坏情况的概率均为0，故常规的平均复杂度根本不具任何参考意义。作为评定算法性能的一种重要尺度，分摊分析（**amortized analysis**）的相关方法与技巧将在后续章节陆续介绍。

■ $O(1)$ 分摊时间

以可扩充向量为例，可以考查对该结构的连续 n 次（查询、插入或删除等）操作，将所有操作中用于内部数组扩容的时间累计起来，然后除以 n 。只要 n 足够大，这一平均时间就是用于扩容处理的分摊时间成本。以下我们将看到，即便排除查询和删除操作而仅考查插入操作，在可扩充向量单次操作中，用于扩容处理的分摊时间成本也不过 $O(1)$ 。

假定数组的初始容量为某一常数 N 。既然是估计复杂度的上界，故不妨设向量的初始规模也为 N ——即将溢出。另外不难看出，除插入操作外，向量其余的接口操作既不会直接导致溢出，也不会增加此后溢出的可能性，因此不妨考查最坏的情况，假设在此后需要连续地进行 n 次 `insert()` 操作， $n \gg N$ 。首先定义如下函数：

```
size(n) = 连续插入n个元素后向量的规模  
capacity(n) = 连续插入n个元素后数组的容量  
T(n) = 为连续插入n个元素而花费于扩容的时间
```

其中，向量规模从 N 开始随着操作的进程逐步递增，故有：

```
size(n) = N + n
```

既然不致溢出，故装填因子绝不会超过100%。同时，这里的扩容采用了“懒惰”策略——只有在的确即将发生溢出时，才不得不将容量加倍——因此装填因子也始终不低于50%。

概括起来，始终应有：

$$\text{size}(n) \leq \text{capacity}(n) < 2 \cdot \text{size}(n)$$

考虑到 N 为常数，故有：

$$\text{capacity}(n) = \Theta(\text{size}(n)) = \Theta(n)$$

容量以2为比例按指数速度增长，在容量达到 $\text{capacity}(n)$ 之前，共做过 $\Theta(\log_2 n)$ 次扩容，每次扩容所需时间线性正比于当时的容量（或规模），且同样以2为比例按指数速度增长。因此，消耗于扩容的时间累计不过：

$$T(n) = 2N + 4N + 8N + \dots + \text{capacity}(n) < 2 \cdot \text{capacity}(n) = \Theta(n)$$

将其分摊到其间的连续 n 次操作，单次操作所需的分摊运行时间应为 $\mathcal{O}(1)$ 。

■ 其它扩容策略

以上分析确凿地说明，基于加倍策略的动态扩充数组不仅可行，而且就分摊复杂度而言效率也足以令人满意。当然，并非任何扩容策略都能保证如此高的效率。比如，早期可扩充向量多采用另一策略：一旦有必要，则追加固定数目的单元。实际上，无论采用的固定常数多大，在最坏情况下，此类数组单次操作的分摊时间复杂度都将高达 $\Omega(n)$ （习题[2-3]）。

2.4.5 缩容

导致低效率的另一情况是，向量的实际规模可能远远小于内部数组的容量。比如在连续的一系列操作过程中，若删除操作远多于插入操作，则装填因子极有可能远远小于100%，甚至非常接近于0。当装填因子低于某一阈值时，我们称数组发生了下溢（underflow）。

尽管下溢不属于必须解决的问题，但在格外关注空间利用率的场合，发生下溢时也有必要适当缩减内部数组容量。代码2.5给出了一个动态缩容 $\text{shrink}()$ 算法：

```
1 template <typename T> void Vector<T>::shrink() { //装填因子过小时压缩向量所占空间
2     if (_capacity < DEFAULT_CAPACITY << 1) return; //不致收缩到DEFAULT_CAPACITY以下
3     if (_size << 2 > _capacity) return; //以25%为界
4     T* oldElem = _elem; _elem = new T[_capacity >>= 1]; //容量减半
5     for (int i = 0; i < _size; i++) _elem[i] = oldElem[i]; //复制原向量内容
6     delete [] oldElem; //释放原空间
7 }
```

代码2.5 向量内部功能 $\text{shrink}()$

可见，每次删除操作之后，一旦空间利用率已降至某一阈值以下，该算法随即申请一个容量减半的新数组，将原数组中的元素逐一搬迁至其中，最后将原数组所占空间交还操作系统。这里以25%作为装填因子的下限，但在实际应用中，为避免出现频繁交替扩容和缩容的情况，可以选用更低的阈值，甚至取作0（相当于禁止缩容）。

与 $\text{expand}()$ 操作类似，尽管单次 $\text{shrink}()$ 操作需要线性量级的时间，但其分摊复杂度亦为 $\mathcal{O}(1)$ （习题[2-4]）。实际上 $\text{shrink}()$ 过程等效于 $\text{expand}()$ 的逆过程，这两个算法相互配合，在不致实质地增加接口操作复杂度的前提下，保证了向量内部空间的高效利用。当然，就单次扩容或缩容操作而言，所需时间的确会高达 $\Omega(n)$ ，因此在对单次操作的执行速度极其敏感的应用场合以上策略并不适用，其中缩容操作甚至可以完全不予考虑。

§ 2.5 常规向量

2.5.1 直接引用元素

与数组直接通过下标访问元素的方式（形如“ $A[i]$ ”）相比，向量ADT所设置的`get()`和`put()`接口都显得不甚自然。毕竟，前一访问方式不仅更为我们所熟悉，同时也更加直观和便捷。那么，在经过封装之后，对向量元素的访问可否沿用数组的方式呢？答案是肯定的。

解决的方法之一就是重载操作符“`[]`”，具体实现如代码2.6所示。

```
1 template <typename T> T& Vector<T>::operator[](Rank r) const //重载下标操作符
2 { return _elem[r]; } // assert: 0 <= r < _size
```

代码2.6 重载向量操作符`[]`

2.5.2 置乱器

■ 置乱算法

可见，经重载后操作符“`[]`”返回的是对数组元素的引用，这就意味着它既可以取代`get()`操作（通常作为赋值表达式的右值），也可以取代`set()`操作（通常作为左值）。例如，采用这种形式，可以简明清晰地描述和实现如代码2.7所示的向量置乱算法。

```
1 template <typename T> void permute(Vector<T>& V) { //随机置乱向量，使各元素等概率出现于每一位置
2     for (int i = V.size(); i > 0; i--) //自后向前
3         swap(V[i - 1], V[rand() % i]); //V[i - 1]与V[0, i)中某一随机元素交换
4 }
```

代码2.7 向量整体置乱算法`permute()`

该算法从待置乱区间的末元素开始，逆序地向前逐一处理各元素。如图2.2(a)所示，对每一个当前元素 $V[i - 1]$ ，先通过调用`rand()`函数在 $[0, i)$ 之间等概率地随机选取一个元素，再令二者互换位置。注意，这里的交换操作`swap()`，隐含了三次基于重载操作符“`[]`”的赋值。

于是如图(b)所示，每经过一步这样的迭代，置乱区间都会向前拓展一个单元。因此经过 $O(n)$ 步迭代之后，即实现了整个向量的置乱。

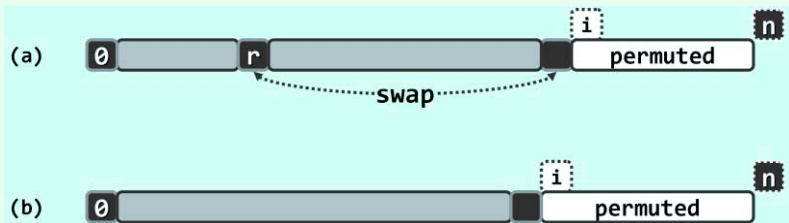


图2.2 向量整体置乱算法`permute()`的迭代过程

在软件测试、仿真模拟等应用中，随机向量的生成都是一项至关重要的基本操作，直接影响到测试的覆盖面或仿真的真实性。从理论上说，使用这里的算法`permute()`，不仅可以枚举出同一向量所有可能的排列，而且能够保证生成各种排列的概率均等（习题[2-6]）。